RELAY HEALTH MONITORING SYSTEM

Full Development Guide — IoT Hardware + Software

ESP32-S3 · Raspberry Pi 4 · MODBUS/RS485 · MQTT · LSTM · Weibull

Stage-by-stage build plan with flowcharts, pinout diagrams, and tables

April 2026

Predictive Maintenance | Contact Resistance | Health Index | RUL Estimation

Table of Contents

Stage 0 — Environment Setup

Before any hardware is connected, set up your complete development environment. Trying to debug firmware while also installing tools wastes hours. Do this first, on Day 0, before any soldering or wiring.

! Golden rule: Build layer by layer and verify each layer independently before integrating with the next. A sensor bug and a MODBUS bug look identical when they're stacked together.

Required tools to install:

Tool	Purpose	Install command / source
Arduino IDE 2.x	ESP32 firmware development	arduino.cc + Espressif ESP32 board package
Python 3.10+	Pi scripts, ML, backend	Pre-installed on Raspberry Pi OS
pymodbus	MODBUS master on Pi	pip install pymodbus
paho-mqtt	MQTT client on Pi	pip install paho-mqtt
FastAPI + uvicorn	REST API + WebSocket	pip install fastapi uvicorn
TensorFlow Lite	Edge ML inference on Pi	pip install tflite-runtime
Mosquitto	MQTT broker on Pi	sudo apt install mosquitto
QModMaster	MODBUS test tool (laptop)	sourceforge.net/projects/qmodmaster
VS Code	IDE for Python + React	code.visualstudio.com
Git	Version control	sudo apt install git (commit every milestone)

Table 1: Development environment tools

Decide your pin assignment before soldering:

Draw the ESP32 pin map on paper and get team agreement before any wire is touched. Changing pins after soldering wastes significant time.

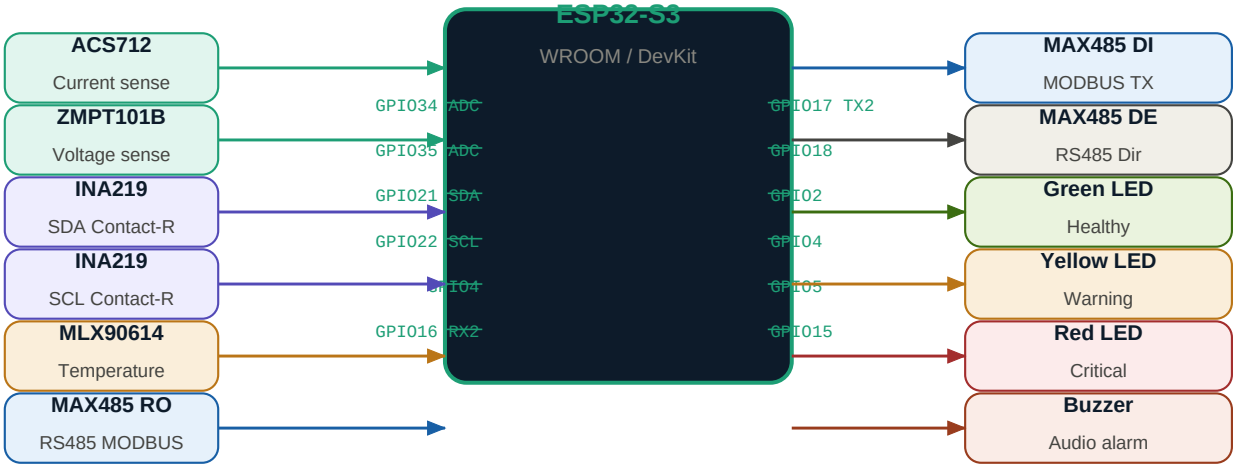


Figure 2: ESP32 pin assignment — sensors (left) and outputs (right)

ESP32 Pin	Module	Direction	Purpose
GPIO 34 (ADC1)	ACS712 OUT	Input	Switched current measurement (AC/DC)
GPIO 35 (ADC1)	ZMPT101B OUT	Input	Switched voltage measurement (AC)
GPIO 21 (SDA)	INA219 SDA	Bidirectional	Contact resistance — I2C data
GPIO 22 (SCL)	INA219 SCL	Output	Contact resistance — I2C clock
GPIO 4	MLX90614 SDA	Bidirectional	Non-contact temperature sensor
GPIO 16 (RX2)	MAX485 RO	Input	MODBUS RTU receive from RS485 bus
GPIO 17 (TX2)	MAX485 DI	Output	MODBUS RTU transmit to RS485 bus
GPIO 18	MAX485 DE/RE	Output	RS485 direction control (HIGH=TX)
GPIO 2	Green LED	Output	Healthy state indicator
GPIO 4	Yellow LED	Output	Warning state indicator
GPIO 5	Red LED	Output	Critical / Imminent state indicator
GPIO 15	Buzzer	Output	Audio alarm (tone() or PWM)
GPIO 21/22	SSD1306 OLED	Output	Local display — HI, cycles, alarm text
GPIO 36 (ADC)	Relay coil GPIO	Input	Relay state detection (switching events)

Table 2: ESP32 pin assignment — finalize before wiring

Stage 1 — IoT Hardware Layer (One Sensor at a Time)

Wire and verify each sensor individually. Do not connect multiple sensors until each one independently produces stable, repeatable readings verified against a reference instrument (multimeter or oscilloscope).

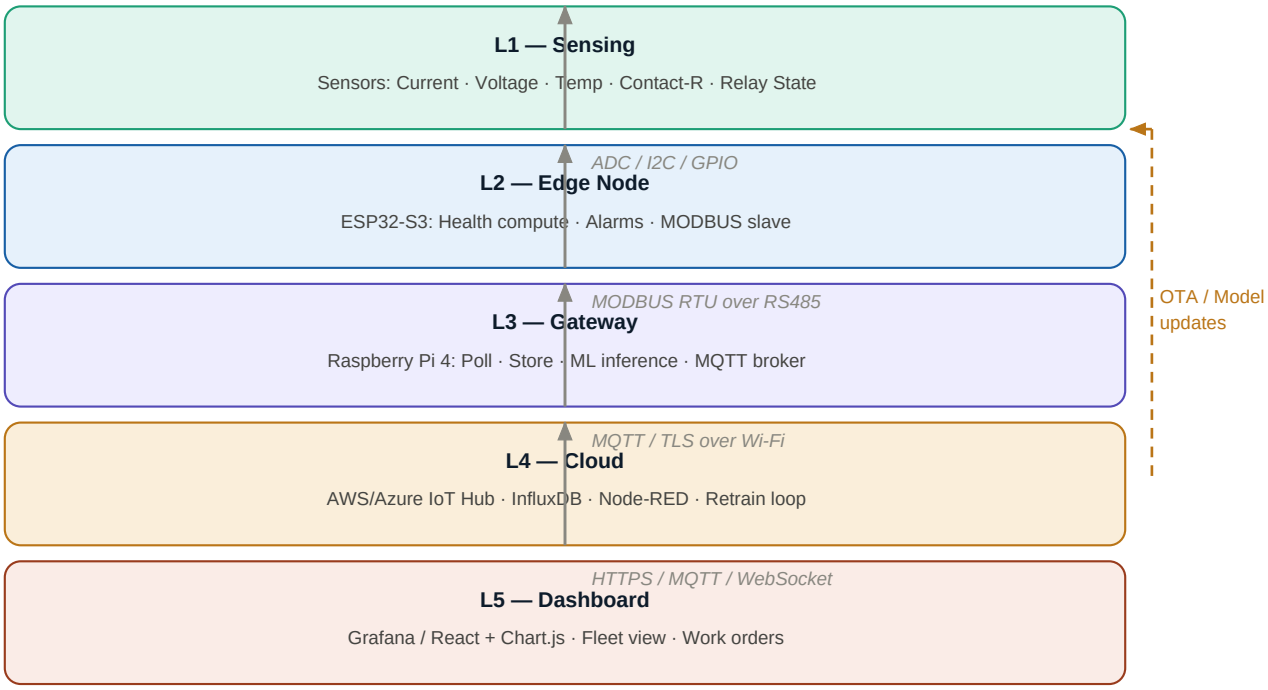


Figure 1: Five-layer system architecture with OTA feedback loop

1.1 — Current Sensor (ACS712)

Wiring: Inline with the relay load circuit. ACS712 needs 5V VCC; its output must be divided to 3.3V before the ESP32 ADC. Use a 3.3kΩ / 6.8kΩ voltage divider.

Verification: Zero load → ADC reads ~2048 (12-bit mid-scale). Known resistive load → reading shifts proportionally. Take 1000 ADC samples per reading and average to reduce noise.

■ Common pitfall: ADC input must never exceed 3.3V. A direct 5V output from ACS712 will damage the ESP32. Always use the voltage divider.

1.2 — Voltage Sensor (ZMPT101B)

Wiring: Connected in parallel with the load (across relay output terminals). Adjust the on-board potentiometer until output is centred at ~1.65V with no load.

Verification: Sample at ≥1 kHz and compute RMS: V_rms = sqrt(mean(sample²)). Compare to multimeter reading. Should match within 5%.

■ Common pitfall: ZMPT101B outputs a bipolar AC signal centred at 1.65V. Negative excursions below 0V will be clipped by the ADC — adjust the offset trim pot until the waveform is fully within 0–3.3V range.

1.3 — Temperature (MLX90614)

Wiring: I2C at address 0x5A. 4.7kΩ pull-ups on SDA and SCL to 3.3V. Aim sensor at relay coil or contact area (non-contact IR measurement).

Verification: Point at hand vs cold surface — object temperature changes while ambient stays stable. Use Adafruit MLX90614 library.

■ Common pitfall: The MLX90614 default I2C address is 0x5A. If running multiple sensors on the same bus, use the Adafruit library's address-change function to assign unique addresses before wiring them together.

1.4 — Contact Resistance (INA219) — Most Critical

Wiring: 4-wire (Kelvin) measurement. Inject 10mA through contacts via a 330Ω resistor from 3.3V. Connect INA219 sense pins directly to contact faces (not through the current-carrying wire). R = V_sense / I_inject.

Verification: New relay: 1–5 mΩ. Any reading above 50 mΩ on a new relay indicates sense wire resistance — move clips closer to contact faces.

■ Common pitfall: The sense wires must carry zero current — they only measure voltage. Using a single pair of wires for both current injection and voltage sense adds wire resistance to your measurement. This is the most common mistake.

1.5 — Relay State Detection

Wiring: 10kΩ pull-down on GPIO. Signal transistor or optocoupler driven by relay coil drive signal. GPIO HIGH = relay energised.

Verification: Toggle relay manually with a button. GPIO state follows. Implement 5ms software debounce to avoid counting one physical switch as multiple events.

■ Common pitfall: Without debounce, a single relay closure generates 5–20 false edge detections due to contact bounce. These inflate N_cycles significantly within hours.

Sensor	Module	Interface	Range	Verify Against
Current	ACS712-30A	ADC (GPIO34)	0–30 A	Clamp meter
Voltage	ZMPT101B	ADC (GPIO35)	0–250 V AC	Multimeter
Temperature	MLX90614	I2C (0x5A)	-40–125 °C	Thermometer
Contact-R	INA219	I2C (0x40)	0–200 mΩ	Milliohm meter
Relay State	GPIO + optocoupler	Digital (GPIO36)	HIGH/LOW	Manual toggle

Table 3: Sensor summary with verification method

Stage 2 — ESP32 Firmware Architecture (FreeRTOS)

Do not write a single monolithic loop(). It will block on I2C calls and give you inaccurate timing for cycle counting. Split the work across two FreeRTOS tasks pinned to separate cores. This is the most important firmware architectural decision.

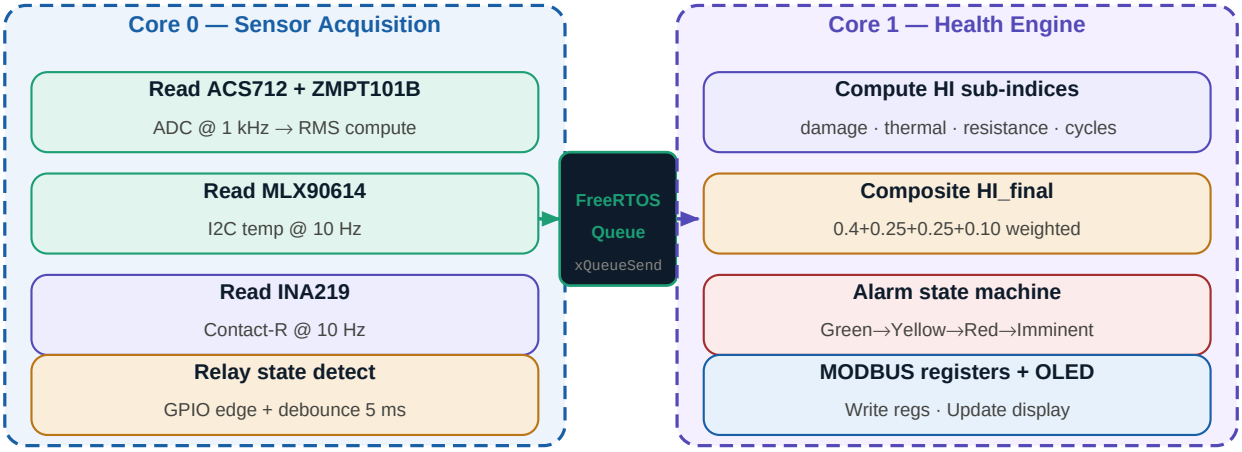


Figure 3: FreeRTOS dual-core task split — Core 0 reads sensors, Core 1 computes health

Task design rules:

- Core 0 task runs at the highest priority (5) and runs every 100ms — it must never be blocked by I2C, MODBUS, or display operations.
- Core 1 task runs at priority 4 and reads from the FreeRTOS Queue using xQueueReceive with a timeout — it blocks until data is available, consuming no CPU while idle.
- The Queue is the only shared data structure between cores. Never access shared variables without a mutex — use xSemaphoreCreateMutex() for any variable read by both cores.
- The OLED display updates at 500ms intervals, not 100ms — writing I2C at 100ms blocks the health computation and introduces jitter into cycle timing.
- Non-volatile storage (NVS) writes happen every 100 switching cycles, not every cycle — flash has limited write endurance (~100,000 cycles per cell).

FreeRTOS task structure (pseudo-code):

```
// Core 0 — pinned to core 0, priority 5
void sensor_task(void *param) {
SensorData data;
while(1) {
data.I_rms = read_acs712_rms(1000); // 1000 samples
data.V_rms = read_zmpt101b_rms(1000);
data.T_coil = mlx.readObjectTempC();
data.R_cont = ina219.read_milliohms();
data.sw_event= detect_relay_edge(); // debounced
xQueueSend(sensor_q, &data;, 0); // non-blocking
vTaskDelay(pdMS_TO_TICKS(100));
}
}
// Core 1 — pinned to core 1, priority 4
```

```
void health_task(void *param) {
SensorData data;
while(1) {
xQueueReceive(sensor_q, &data;, portMAX_DELAY);
if (data.sw_event) {
float E = data.I_rms * data.I_rms * data.V_rms * t_close;
float K = arrhenius_factor(data.T_coil);
W_total += E * K;
N_cycles++;
if (N_cycles % 100 == 0) nvs_save(N_cycles, W_total);
}
float HI = compute_composite_HI(W_total, data.T_coil,
data.R_cont, N_cycles);
update_alarm_state(HI);
write_modbus_registers(HI, N_cycles, data);
update_oled_if_due(HI, N_cycles);
}
}
```

Stage 3 — Health Index Computation Model

The Health Index (HI) is a single number from 0 to 100% representing the relay's remaining life. It is a weighted composite of four independently computed sub-indices, each capturing a different failure mechanism.

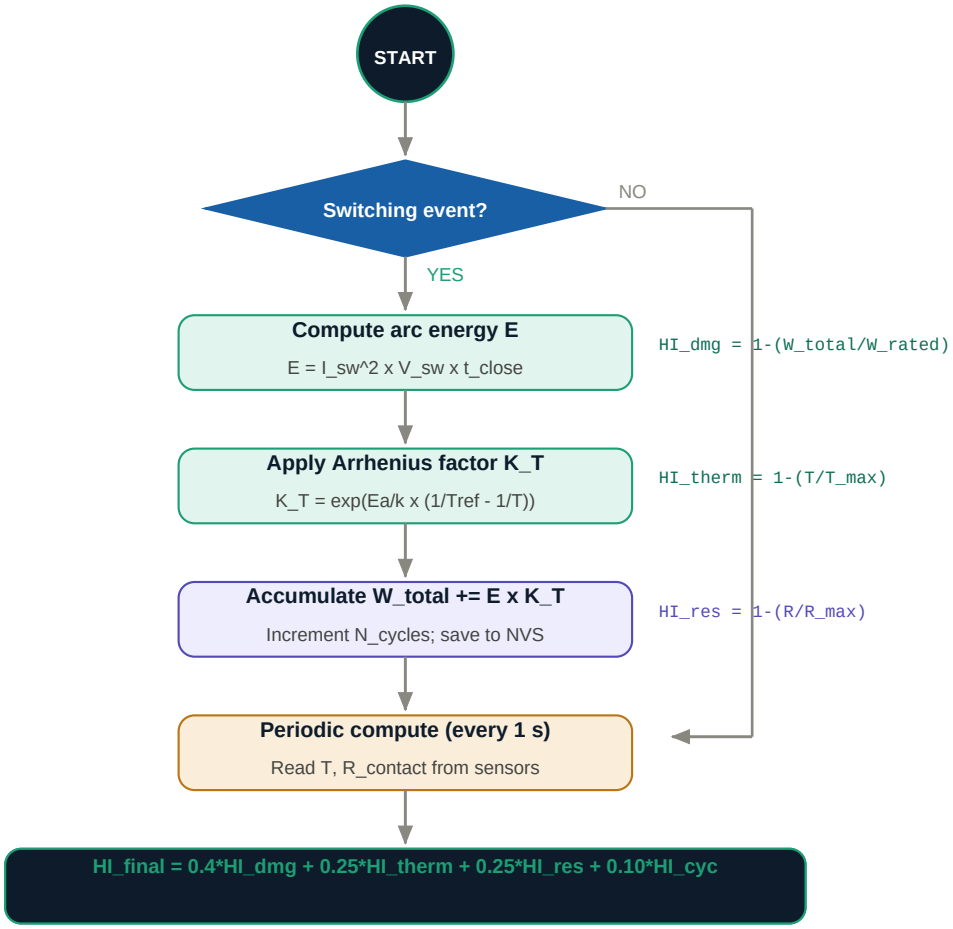


Figure 4: Health Index computation flowchart — runs every second on Core 1

The four sub-indices and their formulas:

Sub-index	Formula	Weight	What it captures
HI_damage	1 - (W_total / W_rated) W = sum of I^2 x V x t x K_T per switch	40%	Arc erosion of contact material — the dominant wear mechanism
HI_thermal	1 - (T_coil / T_max) T_max from relay datasheet	25%	Coil insulation aging — Arrhenius: every 10C halves insulation life
HI_resistance	1 - (R_contact / R_max) R_max = 200 milliohms (critical threshold)	25%	Direct contact surface wear — most sensitive early warning signa
HI_cycles	1 - (N_cycles / N_rated) N_rated from datasheet (e.g. 100,000)	10%	Mechanical fatigue — secondary indicator, crude alone

Table 4: Health Index sub-indices — formulas, weights, and physical meaning

Composite formula:

HI_final = 0.40 × HI_damage + 0.25 × HI_thermal + 0.25 × HI_resistance + 0.10 × HI_cycles

∑ The Arrhenius temperature factor K_T = exp((Ea/k) × (1/T_ref - 1/T)) amplifies arc energy at high temperatures. A relay switching at 85°C accumulates damage 4× faster than at 25°C. Ea = 0.7 eV for contact material, k = 8.617×10^-5 eV/K.

Anomaly overrides (bypass HI thresholds):

Override trigger	Condition	Action
Contact resistance spike	R_contact > 3× baseline value	Immediately force alarm_state = CRITICAL regardless of HI
Bounce duration increase	Bounce time > 1.5× datasheet spec	Force alarm_state = WARNING — indicates mechanical wear
Thermal runaway	dT/dt > 2°C per second	Force alarm_state = CRITICAL — indicates coil short or overload

Table 5: Anomaly overrides — conditions that escalate alarm regardless of HI

Stage 4 — Alarm State Machine

The alarm system has four states. Each state maps directly to a physical output (LED colour and pattern, buzzer behaviour) and a recommended maintenance action. The state machine runs inside health_task on Core 1 after every HI computation.

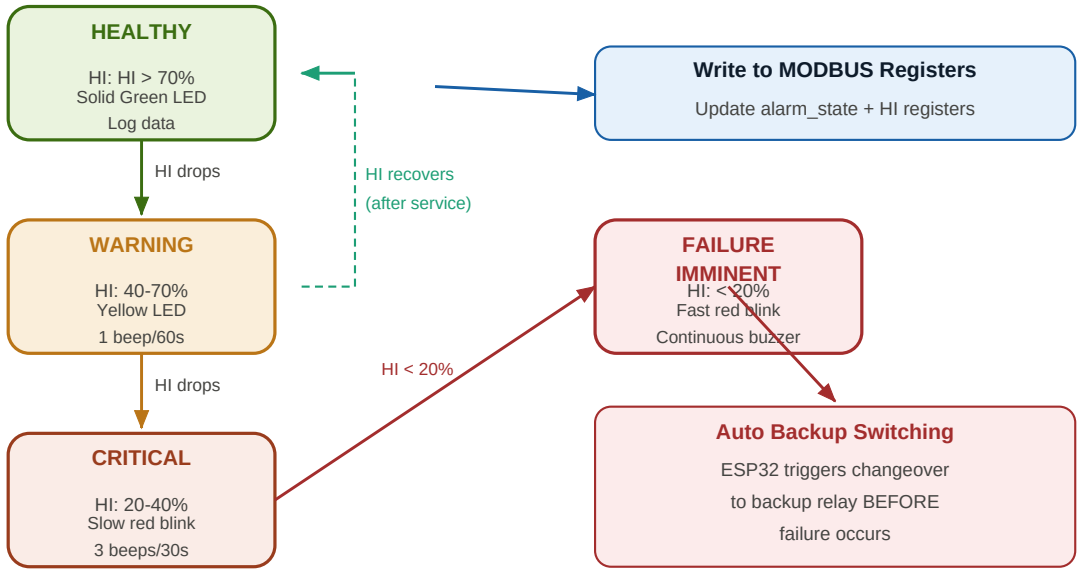


Figure 5: Alert state machine — four health states with transitions and actions

State	HI Range	RUL Estimate	LED	Buzzer	Action
Healthy	> 70%	> 5000 cycles	Solid Green	Silent	Log data, normal operation
Warning	40–70%	1000–5000 cycles	Solid Yellow	1 beep / 60s	Alert dashboard + SMS to technician
Critical	20–40%	100–1000 cycles	Slow blink Red	3 beeps / 30s	Maintenance work order + email alert
Failure Imminent	< 20%	< 100 cycles	Fast blink Red	Continuous	Auto-switch to backup relay + emergency alert

Table 6: Alert state machine — four states with outputs and required actions

! Automatic Backup Switching: When HI drops below 20%, the ESP32 can assert a GPIO to trigger a changeover relay BEFORE failure occurs. This is the difference between predictive maintenance and reactive repair. Wire a spare GPIO to the backup relay coil through a transistor driver.

Stage 5 — MODBUS Register Map

The ESP32 acts as a MODBUS RTU slave at 9600 baud, 8N1, over RS485 via MAX485. Use the ArduinoModbus library. All health data is exposed as Holding Registers at fixed addresses — never change these addresses after the Pi polling script is deployed.

✓ Verification: Before connecting the Pi, use QModMaster on a laptop connected via USB-to-RS485 adapter. All 14 registers should be readable and values should be sane. Do not move to Stage 6 until this passes.

Address	Name	Type	Scale	Description
40001	N_cycles_Hi	UINT16	×1	Operation count — high 16 bits of UINT32
40002	N_cycles_Lo	UINT16	×1	Operation count — low 16 bits of UINT32
40003	HI_final	UINT16	÷1000	Composite health index (0=dead, 1000=new)
40004	HI_damage	UINT16	÷1000	Damage sub-index
40005	HI_thermal	UINT16	÷1000	Thermal sub-index
40006	HI_resist	UINT16	÷1000	Contact resistance sub-index
40007	alarm_state	UINT16	×1	0=Healthy · 1=Warning · 2=Critical · 3=Imminent
40008	T_coil	INT16	÷10	Coil temperature in 0.1°C units
40009	R_contact	UINT16	÷100	Contact resistance in 0.01 mΩ units
40010	I_last_sw	UINT16	÷100	Current at last switch event in 0.01 A units
40011	V_last_sw	UINT16	÷10	Voltage at last switch event in 0.1 V units
40012	W_total	UINT16	×1	Accumulated arc energy (arbitrary normalised units)
40013	dT_dt	INT16	÷10	Temperature rate of change in 0.1°C/s
40014	fault_bits	UINT16	bitmap	Bit 0=R_spike · Bit 1=bounce · Bit 2=thermal runaway

Table 7: MODBUS holding register map (all addresses are 4xxxx Holding Registers)

RS485 bus topology:

- One RS485 bus supports up to 32 ESP32 slave nodes — one per relay being monitored.
- Each ESP32 is assigned a unique MODBUS slave address (1–32) configured at boot from NVS.
- Use a 120Ω termination resistor at both ends of the RS485 cable for runs > 3 metres.
- Maximum bus cable length: 1200 metres at 9600 baud (reduces to 100m at 115200 baud).
- The Raspberry Pi Master polls each slave in sequence: slave 1, slave 2, ... slave N, repeat.

Stage 6 — Raspberry Pi Gateway Software

The Pi is the bridge between real-time edge nodes and the cloud. It must be set up in strict order — each module verified before the next is added. Running all modules together before any are individually verified makes debugging nearly impossible.

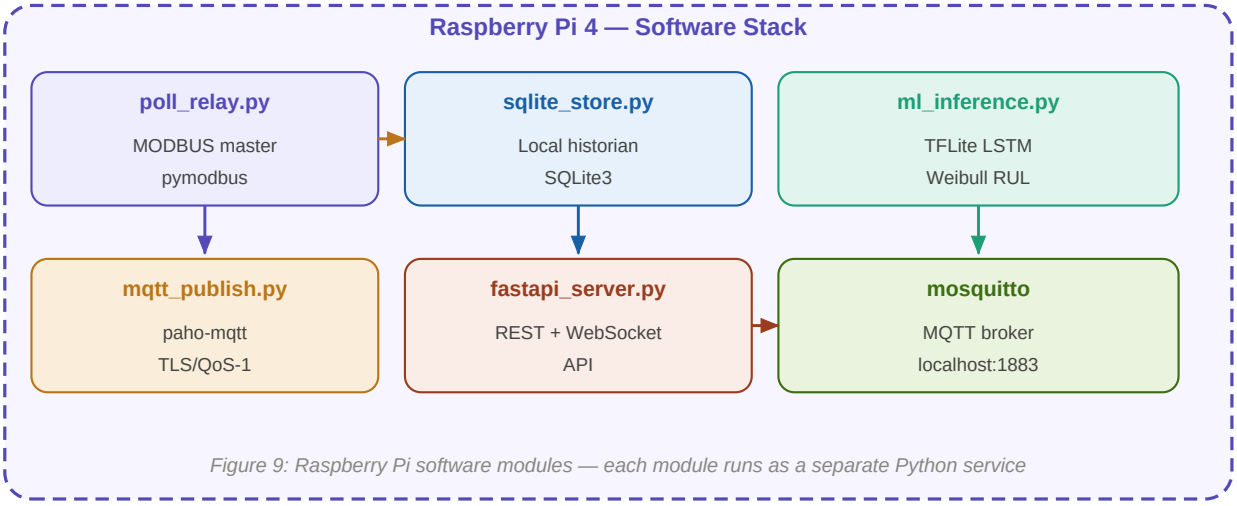


Figure 9: Raspberry Pi software modules — each module runs as a separate Python service

Step 6.1 — MODBUS polling script

Start here. Write only this. Verify that all 14 registers from every slave node are readable and sane before writing to SQLite.

- pymodbus.client.ModbusSerialClient on /dev/ttyUSB0
- Poll each slave address in a loop every 5 seconds
- Print values to console — verify against QModMaster readings
- Only proceed when all registers read consistently for 10+ minutes

Step 6.2 — SQLite local storage

Add database writes after MODBUS polling is confirmed working. Use two tables: readings (every poll) and events (alarm changes, threshold crossings).

- Never overwrite — always INSERT (append). ML needs full time-series history.
- Index on (device_id, ts) for fast range queries
- Store raw register values AND computed HI — never recompute from lost raw data
- Flush WAL every 60 seconds: conn.execute('PRAGMA wal_checkpoint')

Step 6.3 — MQTT publishing

After SQLite writes are confirmed, add MQTT publishing to the same loop. Publish to topic inv/{device_id}/rul with QoS 1 and retain=True.

- Use paho-mqtt with TLS for any cloud broker
- QoS 1 = at-least-once delivery — safe for health data (idempotent inserts)
- retain=True means a new dashboard subscriber immediately gets the last value
- Test with mosquitto_sub -t 'inv/#' -v on the same Pi

Step 6.4 — FastAPI backend

Build last. The API serves the dashboard and WebSocket clients. It subscribes to MQTT and pushes updates to all connected browsers.

- GET /api/fleet — current health of all devices (from in-memory dict)
- GET /api/device/{id}/history?limit=200 — SQLite query
- WebSocket /ws — push every MQTT message to all connected browsers
- POST /api/alerts/{id}/acknowledge — mark alert handled in DB

Stage 7 — MQTT Communication Layer

MQTT is the messaging backbone between the Pi gateway and the cloud. Choose topic names carefully — they are permanent. Changing them later breaks all cloud subscribers.

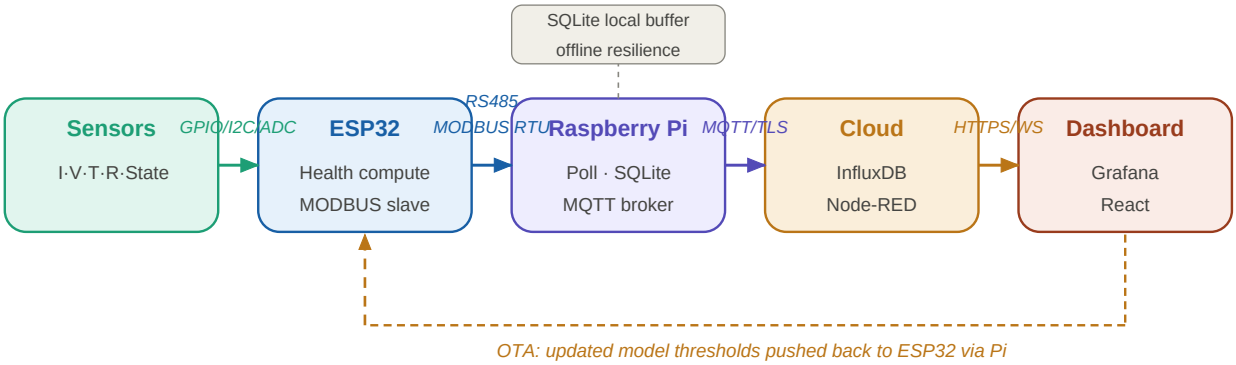


Figure 6: End-to-end data pipeline — sensors to dashboard with OTA feedback

Topic pattern	Publisher	Payload	QoS	Retained
inv/{id}/rul	Pi gateway	Full health JSON (HI, alarm, T, R, cycles)	1	Yes
inv/{id}/alarm	Pi gateway	Alarm level 0–3 + fault bitmap	1	Yes
inv/{id}/raw	Pi gateway	Raw sensor readings (fast telemetry)	0	No
inv/{id}/event	Pi gateway	Switching events with I, V, timestamp	1	No
fleet/summary	Cloud	Aggregated fleet health overview	1	Yes
edge/{id}/config	Cloud	OTA threshold update → Pi → ESP32 NVS	1	No

Table 8: MQTT topic structure — publisher, payload, QoS, and retain settings

JSON payload format (inv/{id}/rul):

```
{
"device_id": "relay_K1",
"ts": 1718000000,
"hi_final": 0.742,
"hi_damage": 0.810,
"hi_thermal": 0.920,
"hi_resistance": 0.680,
"alarm_state": 0,
"t_coil": 42.5,
"r_contact": 8.3,
"i_last_sw": 12.4,
"n_cycles": 14823,
"rul_cycles": 49000
}
```

Stage 8 — ML Pipeline (LSTM + Weibull)

Do not start this stage until you have at least 2–3 days of real sensor data in SQLite. The ML model is only as good as its training data. For the hackathon, synthetic data is acceptable — but label it clearly so judges know.

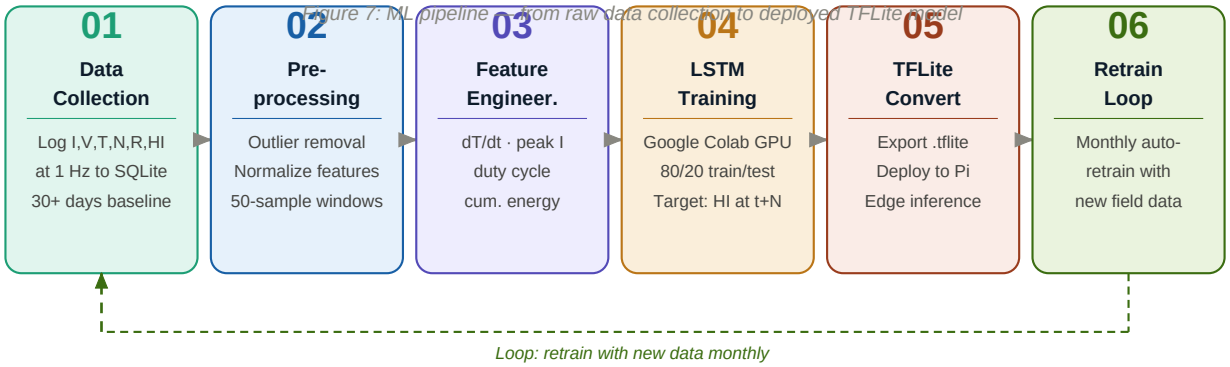


Figure 7: ML pipeline — from raw data collection to deployed TFLite model

8.1 — LSTM Autoencoder (Anomaly Detection)

The LSTM autoencoder learns what a normal degradation trajectory looks like. It is trained on healthy relay sequences and flags anomalies when reconstruction error exceeds threshold. This fires 10–20 data points before any rule-based threshold triggers — that early-detection gap is your AI value story.

- Input: sliding window of 50 timesteps × 10 features (HI sub-indices, T, R, I, dT/dt, resistance slope, switching frequency).
- Architecture: Encoder LSTM(32) → bottleneck → Decoder LSTM(32) → TimeDistributed Dense(10).
- Training target: reconstruct the input sequence (autoencoder — no labels needed).
- Anomaly threshold: mean reconstruction error + 3 standard deviations on the healthy training set.
- Train on Google Colab (GPU). Export to TFLite. Deploy the .tflite file to the Pi for inference-only.

8.2 — Weibull Survival Model (Probabilistic RUL)

The Weibull model gives a probability distribution over remaining life rather than a point estimate. 'This relay has an 85% probability of surviving 8,000 more operations' is far more useful for planning than 'RUL = 8,000 cycles'.

- Shape parameter β (beta): from relay datasheet or literature. β > 1 means wear-out failure (increasing hazard). Typical relay contacts: β ≈ 2.5–3.5.
- Scale parameter η (eta): fitted from B10 life data (the cycle count at which 10% of units fail). Most relay datasheets publish B10.
- RUL at 50th percentile (median): t_50 = η × (ln 2)^(1/β)
- RUL at 85th percentile (conservative): t_85 = η × (-ln 0.15)^(1/β)
- As fleet data accumulates, refit β and η from actual field failures to replace the datasheet estimates.

8.3 — Feature engineering (key derived features):

Feature	Formula	Why it matters
dT_dt	dT/dt = (T_now - T_prev) / dt	Early sign of thermal runaway — spikes before T_max is reached
resistance_slope	Linear regression slope over last 100 readings	Rising trend predicts contact failure before threshold is crossed
switching_frequency	N_switches in last 60s	High frequency = accelerated wear; low frequency = normal

Feature	Formula	Why it matters
peak_current_ratio	I_max_in_window / I_rated	Captures overload events that dominate arc energy accumulation
cumulative_energy	W_total (already computed on ESP32)	Direct damage metric — the primary input to HI_damage

Table 9: Engineered features for LSTM training — these outperform raw sensor values alone

Stage 9 — Dashboard Development

The dashboard is what judges and operators see. Build it after the data pipeline is working — live data makes the dashboard convincing. Use React + Recharts for the web frontend, or Grafana for a faster setup if your team prefers no-code dashboarding.

PRIORITY 1

Fleet Overview Card Grid

One card per relay. Shows relay ID, large health percentage colored green/amber/red, alarm state text, and RUL in cycles or days. This is what a supervisor sees first.

PRIORITY 2

Health Index Timeline

Line chart (last 7 days) of HI_final with four sub-index lines lighter. Vertical markers at alarm state changes. Red dots where LSTM anomaly fired. The gap between LSTM fire and rule-based threshold is your AI advantage.

PRIORITY 3

Contact Resistance Trend

Line chart of R_contact in mΩ over time. Horizontal reference lines at 50 mΩ (warning) and 200 mΩ (critical). This is the most sensitive early-warning visualization.

PRIORITY 4

Current vs Damage Scatter

Scatter plot of switching events: I_switch on x-axis, arc energy on y-axis. Points colored by temperature. Shows why high-current switching is disproportionately damaging.

PRIORITY 5

Work Order Panel

Auto-populates when alarm_state >= 2. Shows relay ID, current HI, predicted failure date from Weibull median RUL, and an acknowledge button.

BONUS

Weibull Survival Curve

Probability of survival vs remaining cycles. Shows 50th, 85th, and 95th percentile bands. Demonstrates probabilistic RUL vs point estimate to sophisticated audiences.

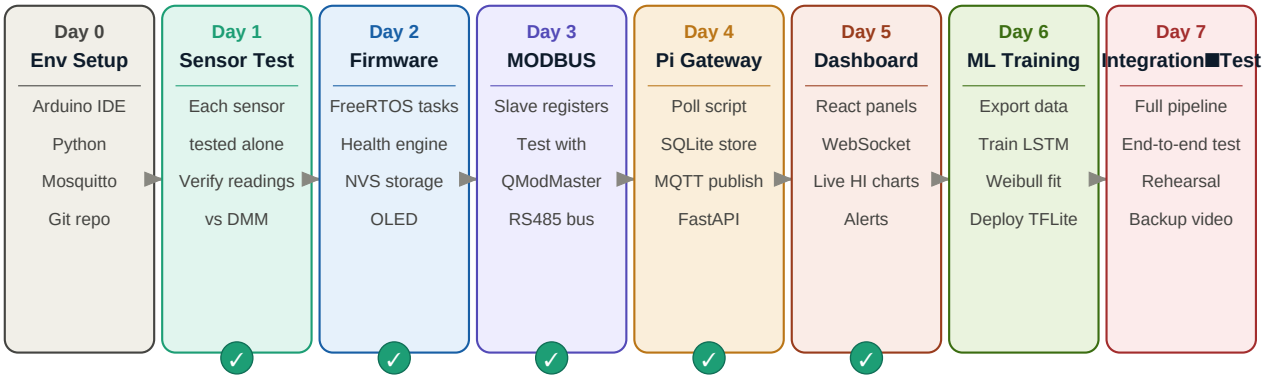
Technology options:

Option	Stack	Setup time	Best for
React + Recharts	React, Recharts, WebSocket client	2–3 days	Custom UI, hackathon demo polish
Grafana + InfluxDB	Grafana OSS, InfluxDB v2, Telegraf	2–4 hours	Quick dashboard, non-React teams
Node-RED Dashboard	Node-RED, node-red-dashboard plugin	1–2 hours	Fastest prototype, limited flexibility

Table 10: Dashboard technology options — choose based on team skill and time

Stage 10 — Build Sequence & Verification Checkpoints

Follow this sequence strictly. The verify checkpoint (✓) at each stage is a go/no-go gate — do not proceed until it passes. Most project failures happen because teams skip verification and stack undetected bugs.



Verify gates (✓): each layer must pass before starting next

Figure 8: Day-by-day development sequence with verification checkpoints

Stage	What to build	Verify checkpoint (MUST PASS before next stage)
0 — Env	All tools installed, Git repo, pin map finalised	Can flash a blink sketch to ESP32. Python imports work on Pi. Git commit pushed.
1 — Sensors	Each sensor wired and tested individually	ACS712 reads known current ±5%. ZMPT101B RMS matches multimeter. MLX90614 res
2 — Firmware	FreeRTOS tasks, health engine, NVS, OLED, slave HI	OLED shows live HI. LED changes colour as you manually change input values. NVS su
3 — MODBUS	MODBUS slave registers on ESP32, MAX485 wiring	QModMaster on laptop reads all 14 registers. Values match OLED display. Multi-node: sl
4 — Pi poll	MODBUS master poll script, SQLite write	SQLite grows every 5 seconds. SELECT * FROM readings shows sane values for all nod
5 — MQTT	MQTT publish from Pi, broker on Pi	mosquitto_sub -t 'inv/#' shows JSON arriving every 5 seconds. Payload values match SQ
6 — API	FastAPI, WebSocket, /api/fleet endpoint	Browser fetches /api/fleet and shows JSON. WebSocket client (wscat) receives push upd
7 — Dashboard	React or Grafana panels with live data	HI gauge updates in browser without page refresh. Alarm state change is visible within 10
8 — ML	LSTM trained, deployed to Pi, Weibull fitted	rul_inference.py loads .tflite and runs inference without errors. Anomaly flag appears in A

Table 11: Full build sequence with mandatory verification checkpoints

! The most common failure mode: trying to debug sensor readings, health computation, MODBUS framing, and the Pi script all at once. Each layer must be independently verified before integration. That discipline is what separates a working demo from a system that almost works.

Team responsibility matrix:

Role	Stages owned	Day target	Deliverable
Hardware Lead	0, 1, 2, 3	Day 3	All sensors verified, MODBUS readable from laptop
Backend Lead	4, 5, 6	Day 5	Pi pipeline live, API serving JSON, WebSocket push working
ML Lead	8	Day 6	LSTM deployed on Pi, Weibull endpoint returning RUL
Frontend Lead	9	Day 6	Dashboard live with real data, all 5 panels visible

Role	Stages owned	Day target	Deliverable
Full Team	Integration	Day 7	End-to-end test, demo rehearsal, backup video recorded

Table 12: Team responsibility matrix — stages, day targets, and deliverables